

Install Libraries

```
!pip install -q transformers datasets scikit-learn accelerate gradio
```

Import and Setup

```
import random
import numpy as np
import torch

from datasets import load_dataset
from transformers import (
    AutoTokenizer,
    AutoModelForSequenceClassification,
    TrainingArguments,
    Trainer,
    DataCollatorWithPadding
)

from sklearn.metrics import accuracy_score, f1_score
```

```
#Reproducibility
SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(SEED)
```

```
!pip install -U datasets huggingface_hub
```

```
Requirement already satisfied: datasets in /usr/local/lib/python3.12/dist-packages (5.0.1)
Requirement already satisfied: huggingface_hub in /usr/local/lib/python3.12/dist-packages (1.27.0)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from datasets) (3.29.7)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from datasets) (2.0.2)
Requirement already satisfied: pyarrow>=21.0.0 in /usr/local/lib/python3.12/dist-packages (from datasets) (25.0.1)
Requirement already satisfied: dill<0.4.2,>=0.3.0 in /usr/local/lib/python3.12/dist-packages (from datasets) (0.3.8)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (from datasets) (2.2.2)
Requirement already satisfied: requests>=2.32.2 in /usr/local/lib/python3.12/dist-packages (from datasets) (2.32.4)
Requirement already satisfied: httpx<1.0.0 in /usr/local/lib/python3.12/dist-packages (from datasets) (0.28.1)
Requirement already satisfied: tqdm>=4.66.3 in /usr/local/lib/python3.12/dist-packages (from datasets) (4.67.3)
Requirement already satisfied: xxhash in /usr/local/lib/python3.12/dist-packages (from datasets) (3.8.1)
Requirement already satisfied: multiprocessing<0.70.20 in /usr/local/lib/python3.12/dist-packages (from datasets) (0.70.16)
Requirement already satisfied: fsspec<=2026.6.0,>=2023.1.0 in /usr/local/lib/python3.12/dist-packages (from fsspec[http]<=2026.6.0) (2026.6.0)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from datasets) (26.2)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from datasets) (6.0.3)
Requirement already satisfied: click<9.0.0,>=8.4.2 in /usr/local/lib/python3.12/dist-packages (from huggingface_hub) (8.4.2)
Requirement already satisfied: hf-xet<2.0.0,>=1.5.2 in /usr/local/lib/python3.12/dist-packages (from huggingface_hub) (1.6.0)
Requirement already satisfied: typing-extensions>=4.1.0 in /usr/local/lib/python3.12/dist-packages (from huggingface_hub) (4.16.0)
Requirement already satisfied: aiohttp!=4.0.0a0,!4.0.0a1 in /usr/local/lib/python3.12/dist-packages (from fsspec[http]<=2026.6.0) (4.14.2)
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-packages (from httpx<1.0.0->datasets) (4.14.2)
Requirement already satisfied: certifi in /usr/local/lib/python3.12/dist-packages (from httpx<1.0.0->datasets) (2026.6.17)
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from httpx<1.0.0->datasets) (1.0.9)
Requirement already satisfied: idna in /usr/local/lib/python3.12/dist-packages (from httpx<1.0.0->datasets) (3.18)
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpcore==1.*->httpx<1.0.0->datasets) (0.14.0)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2->datasets) (3.4.0)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2->datasets) (2.3.0)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas->datasets) (2.9.0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas->datasets) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas->datasets) (2026.3)
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!4.0.0a1->fsspec) (2.5.0)
Requirement already satisfied: aiosignal>=1.4.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!4.0.0a1->fsspec) (1.4.0)
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!4.0.0a1->fsspec) (25.1.0)
Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!4.0.0a1->fsspec) (1.5.0)
Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!4.0.0a1->fsspec) (6.1.0)
Requirement already satisfied: propcache>=0.2.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!4.0.0a1->fsspec) (0.2.0)
Requirement already satisfied: yarl<2.0,>=1.17.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!4.0.0a1->fsspec) (1.18.3)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas->dataset
```

```
from datasets import load_dataset

dataset = load_dataset("stanfordnlp/imdb")
```

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster d
 WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to en

```
print(dataset)
```

```
DatasetDict({
  train: Dataset({
    features: ['text', 'label'],
    num_rows: 25000
  })
  test: Dataset({
    features: ['text', 'label'],
    num_rows: 25000
  })
  unsupervised: Dataset({
    features: ['text', 'label'],
    num_rows: 50000
  })
})
```

```
sample = dataset["train"][0]
print("Label",sample["label"],"(0 = negative, 1 = positive)")
print("\nReview",sample["text"][:500],"...")
```

Label 0 (0 = negative, 1 = positive)

Review I rented I AM CURIOUS-YELLOW from my video store because of all the controversy that surrounded it when it was first rele

```
TRAIN_SIZE = 2000
TEST_SIZE = 250

small_train = dataset["train"].shuffle(seed=SEED).select(range(TRAIN_SIZE))
small_test= dataset["train"].shuffle(seed=SEED).select(range(TRAIN_SIZE))

# split the training data into trian + validation
split = small_train.train_test_split(test_size=0.1, seed=SEED)
train_ds = split["train"]
val_ds = split["test"]

print("Train size:",len(small_train))
print("val size",len(val_ds))
print("Test size",len(small_test))
```

Train size: 2000
 val size 200
 Test size 2000

```
MODEL_NAME = "bert-base-uncased"
MAX_LENGTH = 256

tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)

def tokenize(batch):
    return tokenizer(batch["text"], max_length=MAX_LENGTH, truncation=True)

train_tokenized = train_ds.map(tokenize,batched = True)
val_tokenized = val_ds.map(tokenize,batched = True)
test_tokenized = small_test.map(tokenize,batched = True)
```

Map: 100% 200/200 [00:01<00:00, 154.62 examples/s]

Map: 100% 2000/2000 [00:08<00:00, 227.75 examples/s]

Load pre-trained BERT for Classification

```
model = AutoModelForSequenceClassification.from_pretrained(
    MODEL_NAME,
    num_labels=2
)
```

Loading weights: 100% 199/199 [00:00<00:00, 1642.56it/s]

[transformers] **BertForSequenceClassification** LOAD REPORT from: bert-base-uncased

Key	Status
cls.predictions.bias	UNEXPECTED
cls.predictions.transform.dense.weight	UNEXPECTED
cls.predictions.transform.LayerNorm.bias	UNEXPECTED
cls.seq_relationship.weight	UNEXPECTED
cls.predictions.transform.dense.bias	UNEXPECTED
cls.predictions.transform.LayerNorm.weight	UNEXPECTED
cls.seq_relationship.bias	UNEXPECTED
classifier.bias	MISSING
classifier.weight	MISSING

Notes:

- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING: those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.

Metrics and Data Collator

```
def compute_metrics(eval_pred):
    logits, label = eval_pred
    preds = np.argmax(logits, axis=1)
    return {
        "accuracy": accuracy_score(label, preds),
        "f1": f1_score(label, preds)
    }
```

```
data_collator = DataCollatorWithPadding(tokenizer=tokenizer)
```

```
training_args = TrainingArguments(
    output_dir = "./bert_imdb_output",
    eval_strategy="epoch",
    learning_rate = 2e-5,
    per_device_train_batch_size = 8,
    per_device_eval_batch_size = 8,
    num_train_epochs = 2,
    weight_decay = 0.01,
    fp16=torch.cuda.is_available(),
    report_to="none",
    seed=SEED,
    logging_steps=50,
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_tokenized,
    eval_dataset=val_tokenized,
    processing_class=tokenizer,
    data_collator=data_collator,
    compute_metrics=compute_metrics,
)
```

```
trainer.train()
```

```
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:775: UserWarning: 'pin_memory' argument is set as true but
super().__init__(loader)
```

[450/450 2:58:10, Epoch 2/2]

Epoch	Training Loss	Validation Loss	Accuracy	F1
1	0.394122	0.303857	0.870000	0.857143
2	0.357137	0.408829	0.895000	0.888889

```
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:775: UserWarning: 'pin_memory' argument is set as true but
super().__init__(loader)
```

Writing model shards: 100%

1/1 [00:04<00:00, 4.27s/it]

```
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:775: UserWarning: 'pin_memory' argument is set as true but
super().__init__(loader)
```

TrainOutput(global_step=450, training_loss=0.383917908138699, metrics={'train_runtime': 10751.5005, 'train_samples_per_second': 0.335, 'train_steps_per_second': 0.042, 'total_flos': 473271010828800.0, 'train_loss': 0.383917908138699, 'epoch': 2.0})

```
final_results = trainer.evaluate(eval_dataset=test_tokenized)
print("final_results:")
for k,v in final_results.items():
    if isinstance(v,float):
        print(f"{k}: {v:.4f}")
```

```
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:775: UserWarning: 'pin_memory' argument is set as true but
super().__init__(loader)
```

[199/250 20:07 < 05:10, 0.16 it/s]

```
id2label = {0: "negative", 1: "positive"}

def predict_sentiment(text):
    inputs = tokenizer(
        text,
        max_length=MAX_LENGTH,
        truncation=True,
        return_tensors="pt",
    ).to(model.device)

    model.eval()
    with torch.no_grad():
        logits = model(**inputs).logits

    probs = torch.softmax(logits,dim=-1)[0]
    pred_id = torch.argmax(probs).item()

    return {
        "label": id2label[pred_id],
        "confidence": round(probs[pred_id].item(),4),
    }

reviews = [
    "This movie was absolutely fanatastic.Brilliant acting and a gripping story.",
    "I hated this film. It was boring,too long , and baldy wrtten.",
    "It was okay - some good moments, but overall forgettable.",
]

for r in reviews:
    print(predict_sentiment(r), "->", r[:60], "...")
```

```
SAVE_DIR = "./imdb_bert"
```

```
trainer.save_model(SAVE_DIR)
tokenizer.save_pretrained(SAVE_DIR)
```

```
print("Saved model and tokenizer to:",SAVE_DIR)
```

```
import torch
import gradio as gr
from transformers import AutoTokenizer, AutoModelForSequenceClassification

SAVE_DIR = "./imdb_bert"
```

```

# Reload tokenizer and model
reloaded_tokenizer = AutoTokenizer.from_pretrained(SAVE_DIR)
reloaded_model = AutoModelForSequenceClassification.from_pretrained(SAVE_DIR)

# Select device
device = "cuda" if torch.cuda.is_available() else "cpu"

reloaded_model.to(device)
reloaded_model.eval()

print("Model and tokenizer reloaded on:", device)

# Labels
id2label = {
    0: "negative",
    1: "positive"
}

MAX_LENGTH = 256

def predict_sentiment(text):
    inputs = reloaded_tokenizer(
        text,
        return_tensors="pt",
        truncation=True,
        max_length=MAX_LENGTH
    ).to(device)

    with torch.no_grad():
        logits = reloaded_model(**inputs).logits
        probs = torch.softmax(logits, dim=-1)[0]

    pred_id = torch.argmax(probs).item()

    return id2label[pred_id], probs[pred_id].item()

def classify_review(review):
    if not review.strip():
        return "Please enter a review."

    label, confidence = predict_sentiment(review)

    emoji = "✅" if label == "positive" else "❌"

    return f"{emoji} **{label.upper()}** (confidence: {confidence * 100:.2f}%)"

# Gradio interface
demo = gr.Interface(
    fn=classify_review,

    inputs=gr.Textbox(
        lines=5,
        placeholder="Type a movie review here...",
        label="Movie Review"
    ),

    outputs=gr.Markdown(
        label="Prediction"
    ),

    title="IMDB SENTIMENT CLASSIFIER",

    description=(
        "Fine-tuned BERT predicting whether a movie review "
        "is positive or negative."
    ),

    flagging_mode="never",

    examples=[
        [
            "This movie was absolutely fantastic. Brilliant acting and a gripping story."
        ],
        [

```

```
],
[
    "It was okay - some good moments, but overall forgettable."
],
],
)

demo.launch(share=True)
```

Loading weights: 100% 201/201 [00:00<00:00, 2660.74it/s]

Model and tokenizer reloaded on: cpu
Colab notebook detected. To show errors in colab notebook, set debug=True in launch()
* Running on public URL: <https://d280f2c91c8c790284.gradio.live>

This share link is temporary and will last for up to 1 week (best effort). For free permanent hosting and GPU upgrades, run `gra

IMDB SENTIMENT CLASSIFIER

Fine-tuned BERT predicting whether a movie review is positive or negative.

Movie Review

it was boring

Clear

Submit

✖ NEGATIVE (confidence: 80.14%)

☰ Examples

- This movie was absolutely fantastic. Brilliant acting and a ...
- I hated this film. It was boring, too long, and badly writte...
- It was okay - some good moments, but overall forgettable.

Heavily via ADI 🦄 • Built with Gradio 🍷 • Settings ⚙️

Next steps: [🚀 Deploy to Cloud Run](#)

◆ Gemini

```
import torch
import gradio as gr
from transformers import AutoTokenizer

SAVE_DIR = "./imdb_bert"

reloaded_tokenizer = AutoTokenizer.from_pretrained(SAVE_DIR)
reloaded_model = AutoModelForSequenceClassification.from_pretrained(SAVE_DIR)

device = "cuda" if torch.cuda.is_available() else "cpu"
reloaded_model.to(device)
reloaded_model.eval()

print("Model and token reloaded on:", device)

id2label = {0: "negative", 1: "positive"}
MAX_LENGTH = 256

def predict_sentiment(text):
    inputs = reloaded_tokenizer(
        text,
        return_tensors="pt",
        truncation=True,
        max_length=MAX_LENGTH,
    ).to(device)
    logits = reloaded_model(**inputs)
```

```

        probs = torch.softmax(logits, dim=-1)
        pred_id = torch.argmax(probs).item()
    with torch.no_grad():
        logits = reloaded_model(**inputs)
        probs = torch.softmax(logits, dim=-1)
        pred_id = torch.argmax(probs).item()

    return id2label[pred_id], probs[pred_id].item()

def classify_review(review):
    if not review.strip():
        return "please enter a review"
    label, confidence = predict_sentiment(review)
    emoji = "👍" if label == "positive" else "👎"
    return f"{emoji} {label.upper()} {confidence:.2f}"
    return f"{emoji} {label.upper()} {confidence:.2f}"

demo=gr.Interface(
    fn=classify_review,
    inputs=gr.Textbox(lines=5,
                        placeholder="Type your review here",
                        label="Movie Review"),
    outputs=gr.Markdown(label="Prediction",
                        title="IMDB SENTIMENT CLASSIFIER",
                        description="Fine-tuned BERT pre-trained model for sentiment classification",
                        flagging_mode="never",
                        examples=[
                            ["This movie was absolutely amazing! I loved it."],
                            ["I hated this film. It was terrible."],
                            ["It was okay - some good points but overall not great."],
                        ],
    ),
    demo.launch(share=True)

demo=gr.Interface(
    fn=classify_review,
    inputs=gr.Textbox(lines=5,
                        placeholder="Type your review here",
                        label="Movie Review"),
    outputs=gr.Markdown(label="Prediction",
                        title="IMDB SENTIMENT CLASSIFIER",
                        description="Fine-tuned BERT pre-trained model for sentiment classification",
                        flagging_mode="never",
                        examples=[
                            ["This movie was absolutely amazing! I loved it."],
                            ["I hated this film. It was terrible."],
                            ["It was okay - some good points but overall not great."],
                        ],
    ),
    demo.launch(share=True)

```

File ["/tmp/ipykernel_3458/570012479.py"](#), line 27

```

    with torch.no_grad():
        ^
IndentationError: unexpected indent

```

Next steps: [Explain error](#)